# Object Oriented Programming (CSEG1044)

## Unit 02 – Lecture 02: Constructors, ‘this‘, Overloading, ‘static‘, and GC Overview

Tofik Ali

School of Computer Science, UPES Dehradun

February 27, 2026

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Learning Outcomes

- Write constructors and use `this(...)` for constructor chaining.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Learning Outcomes

- Write constructors and use `this(...)` for constructor chaining.
- Overload methods correctly and avoid common mistakes.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Learning Outcomes

- Write constructors and use `this(...)` for constructor chaining.
- Overload methods correctly and avoid common mistakes.
- Use `static` for class-level state/behavior and explain GC basics.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Unit 02: Classes, Inheritance, Packages and Interfaces

- Lecture focus: Constructors, ‘this‘, Overloading, ‘static‘, and GC Overview

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Today: Roadmap

- Constructors (default/parameterized), constructor chaining, ‘this()‘ / ‘this‘

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Today: Roadmap

- Constructors (default/parameterized), constructor chaining, ‘this()‘ / ‘this‘
- Method overloading (rules + examples)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Today: Roadmap

- Constructors (default/parameterized), constructor chaining, ‘this()‘ / ‘this‘
- Method overloading (rules + examples)
- ‘static‘ keyword (fields/methods/blocks) and utility-class pattern

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Today: Roadmap

- Constructors (default/parameterized), constructor chaining, ‘this()‘ / ‘this‘
- Method overloading (rules + examples)
- ‘static‘ keyword (fields/methods/blocks) and utility-class pattern
- Garbage collection overview (what it is / what it isn’t)

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Constructors (why and how)

- Constructor initializes an object and enforces invariants.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Constructors (why and how)

- Constructor initializes an object and enforces invariants.
- If you define any constructor, Java does **not** add a default no-arg one automatically.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Constructors (why and how)

- Constructor initializes an object and enforces invariants.
- If you define any constructor, Java does **not** add a default no-arg one automatically.
- Use `this(...)` for constructor chaining (must be first statement).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# this vs this(...)

- `this` = current object reference (resolves shadowing: `this.name = name;`).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# `this` vs `this(...)`

- `this` = current object reference (resolves shadowing: `this.name = name;`).
- `this(...)` = calls another constructor (reduces duplicate code).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Method overloading (rules)

- Same method name, different parameter list (number/type/order).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Method overloading (rules)

- Same method name, different parameter list (number/type/order).
- Not overloading: changing only return type.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Method overloading (rules)

- Same method name, different parameter list (number/type/order).
- Not overloading: changing only return type.
- Selected at compile time (based on argument types).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# `static` keyword

- Static field is shared across all objects (e.g., counters).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# static keyword

- Static field is shared across all objects (e.g., counters).
- Static method belongs to class: `ClassName.method()`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# static keyword

- Static field is shared across all objects (e.g., counters).
- Static method belongs to class: `ClassName.method()`.
- Static methods cannot access instance fields directly (no `this`).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Garbage collection (realistic view)

- GC reclaims memory of objects that are no longer reachable.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Garbage collection (realistic view)

- GC reclaims memory of objects that are no longer reachable.
- `System.gc()` is a suggestion, not a guarantee.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Garbage collection (realistic view)

- GC reclaims memory of objects that are no longer reachable.
- `System.gc()` is a suggestion, not a guarantee.
- GC does not automatically close files/DB connections (you must close resources).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Example: constructors + static counter (1/2)

```
public class Employee {
    private static int nextId = 1;

    private final int id;
    private final String name;
    private double salary;

    public Employee() { this("NA", 0.0); }

    public Employee(String name, double salary) {
        this.id = nextId++;
        this.name = name;
        this.salary = salary;
    }
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27,

# Example: constructors + static counter (2/2)

```
// continued...
    public static int getNextId() { return nextId; }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Example: method overloading

```
public void raiseSalary(double percent) {
    raiseSalary(percent, 0.0);
}

public void raiseSalary(double percent, double bonus) {
    salary = salary + (salary * percent / 100.0) + bonus;
}
```

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Mini Exercises (2–3 mins)

1. Difference: `this` vs `this(...)`?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Mini Exercises (2–3 mins)

1. Difference: `this` vs `this(...)`?
2. Can we overload by changing only return type?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Mini Exercises (2–3 mins)

1. Difference: `this` vs `this(...)`?
2. Can we overload by changing only return type?
3. Can a static method access instance fields directly?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Key Takeaways

- Constructors create valid objects and can be chained using `this(...)`.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Key Takeaways

- Constructors create valid objects and can be chained using `this(...)`.
- Overloading = same name, different parameters (compile-time).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Key Takeaways

- Constructors create valid objects and can be chained using `this(...)`.
- Overloading = same name, different parameters (compile-time).
- `static` is class-level; GC is automatic but not controllable exactly.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Checkpoint Questions

1. Why is `this(...)` useful in constructors?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Checkpoint Questions

1. Why is `this(...)` useful in constructors?
2. What is one valid and one invalid way to overload a method?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Checkpoint Questions

1. Why is `this(...)` useful in constructors?
2. What is one valid and one invalid way to overload a method?
3. What does garbage collection do (in one sentence)?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Next Lecture Preview

**Next:** Access Modifiers + Packages + Interface Intro

- We will organize code into packages and control visibility.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).

# Next Lecture Preview

**Next:** Access Modifiers + Packages + Interface Intro

- We will organize code into packages and control visibility.
- We will introduce interfaces as "contracts".

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed February 27, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed February 27, 2026).